

further improve the strategy by three computation-specific optimizations. Finally, we also propose efficient algorithms for approximate nearest neighbor search (ANNS) and range search (RS) queries based on the optimized search procedure.

To the best of our knowledge, this is the first work that tackles the fundamental data locality problem of disk-based graph index for the data segment by effective block shuffling algorithms. Our evaluation shows that Starling achieves 43.9× higher throughput with 98% lower query latency than state-of-the-art methods under the same accuracy. The main contributions of this work are:

- We present Starling, an I/O-efficient disk-resident graph index framework with excellent universality that enhances the search efficiency of various graph index algorithms (e.g., Vamana [35], NSG [25], and HNSW [49]) on a data segment (§3, §6.7).
- We design a data layout for the data segment that consists of two components: an in-memory navigation graph (§4.2) and a reordered disk-based graph (§4.1). This data layout reduces the search path length and increases the vertex utilization ratio of each loaded block when searching on the disk-based graph index.
- We prove that the block shuffling problem is NP-hard and has no polynomial time approximation algorithm with a finite approximation factor unless  $P=NP$  (§4.1). Hence, we devise three effective heuristic shuffling algorithms that improve the locality of the disk-based graph index (§4.1).
- We propose a block search strategy that exploits the data locality of the reorganized index layout to reduce disk I/Os (§5.1). Based on this strategy, we also provide three computation-specific optimizations (§5.1) and develop search algorithms for two major types of queries in vector databases (§5.2 and §5.3).
- We implement Starling and evaluate it on four real-world datasets to verify its effectiveness, efficiency, and scalability (§6). We show that search efficiency can be improved significantly by simply adjusting the index layout on the disk.

## 2 PRELIMINARIES

In this section, we formulate two critical queries in vector databases. Then we explain why current methods are inefficient on the data segment and state our optimization goal in this paper.

### 2.1 Two Query Types for HVSS

**K Nearest Neighbor Search (KNNS).** Given a vector dataset  $X = \{x_1, x_2, \dots, x_n\} \subset \mathbb{R}^D$ , a query vector  $q \in \mathbb{R}^D$ , a distance function  $dist(\mathbb{R}^D, \mathbb{R}^D) \rightarrow \mathbb{R}$ , and a positive integer  $k$  ( $0 < k < n$ ), the KNNS problem finds a set  $R_{knn}$  of  $k$  vectors from  $X$  that are closest to  $q$  such that for any  $x_i \in R_{knn}$  and  $x_j \in X \setminus R_{knn}$ ,

$$dist(x_i, q) \leq dist(x_j, q) \quad . \quad (1)$$

The distance function can be Euclidean distance or others.

The KNNS problem requires scanning all the vectors in the dataset, which is impractical for large vector datasets. Hence, most studies try to find approximate results that are close to the exact KNNS results. This is the Approximate Nearest Neighbor Search (ANNS) problem [1, 24, 25, 49]. ANNS uses  $Recall \in [0, 1]$  to measure the accuracy of approximate results. Let  $R'_{knn}$  be the approximate results, then  $Recall$  is

$$Recall = \frac{|R_{knn} \cap R'_{knn}|}{k} \quad . \quad (2)$$

A higher recall means a more accurate result. Many studies have designed different index structures for vector data to improve the trade-off between search efficiency and accuracy.

**Range Search (RS).** Unlike the KNN problem that returns a fixed number of results, the RS problem retrieves all the vectors that are within a given distance  $r$  from the query vector  $q$ . Average precision ( $AP$ )  $\in [0, 1]$  is a metric to measure the accuracy of approximate RS results. Given the exact result  $R_{range}$  and an approximate solution that ensures all the returned results  $R'_{range}$  are within  $r$ , then  $AP$  can be computed as follows:

$$AP = \frac{|R'_{range}|}{|R_{range}|} \quad (3)$$

## 2.2 HVSS on Data Segment.

Vector databases divide large-scale data into multiple segments and distribute them across servers [23, 29]. Each server may manage numerous segments, processing vector queries either in parallel or serially through a query coordinator. Typically, each segment operates within strict memory and disk space constraints [4, 29], often possessing less than 2GB of memory and under 10GB of disk capacity. Data segment facilitates system scalability [29], yet it also poses challenges for HVSS within a data segment. Within this context, our objective is to achieve high search accuracy and efficiency within these space limitations. This necessitates a delicate equilibrium between search performance and the space cost for HVSS. For instance, on the BIGANN dataset [3], each segment might accommodate 33 million vectors, requiring 4GB of storage. Consequently, the remaining space for index storage becomes exceedingly limited. Note that we consistently construct a substantial index to achieve a better accuracy and efficiency trade-off. In the following, we delve into the reasons why current HVSS advancements struggle within the context of data segments.

Mainstream in-memory algorithms such as HNSW [49], NSG [25], and HVS [46] encounter challenges when handling tens of millions of vectors on a segment due to their approach of loading all vectors and index into memory, exceeding memory limitations. While some vector compression methods like LSH [33] and PQ [36] store compressed vectors in memory, the compression process introduces significant errors that notably degrade search accuracy. For instance, the top-1 recall rate of the leading compression method seldom surpasses 0.5 [35]. Disk-based solutions present more promise as they require less memory and achieve high search accuracy. Traditional disk-based methods based on trees are excluded due to their susceptibility to the “curse of dimensionality” when addressing HVSS [45]. Instead, the focus is directed towards two state-of-the-art disk-based methods for HVSS: SPANN [16] and DiskANN [35]. SPANN achieves high search accuracy and efficiency but at the cost of substantial storage space. It may replicate each vector up to eight times, leading to extensive disk overhead that far surpasses the capacity of the data segment when managing tens of millions of vectors. On the other hand, DiskANN follows the graph index algorithm and achieves superior search performance with less disk capacity. It has served as a baseline for HVSS on disk [3]. However, its data layout and search strategy still align with the paradigm of memory-based solutions, necessitating disk I/O for each vertex access. In summary, devising an efficient and accurate solution for HVSS on a data segment remains challenging, involving the management of tens of millions of high-dimensional vectors within limited space.

## 2.3 Our Optimization Objective

We draw two conclusions from our observations. First, the disk-resident graph index proves to be the optimal choice for HVSS on the data segment, offering improved search performance with minimal space overhead [60]. Second, the disk-based graph index can be further optimized for I/O-efficiency in HVSS. Consequently, the HVSS problem on the data segment can be framed as enhancing the I/O-efficiency of disk-resident graph index. Let  $T_{I/O}$ ,  $T_{comp}$ , and  $T_{other}$  represent the I/O time, computation time, and other times (such as data structure maintenance) when conducting

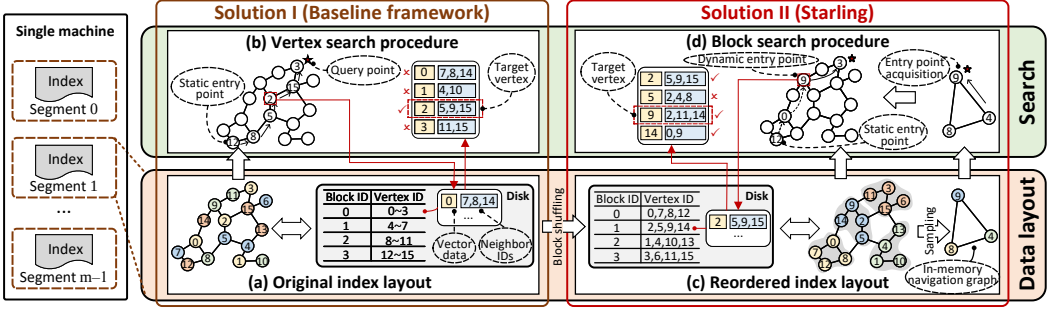


Fig. 2. Illustration of the data layouts and search strategies for the baseline and Starling, respectively.

searches on the disk-based graph, respectively. The total search time can be denoted as

$$T_{total} = T_{I/O} + T_{comp} + T_{other}. \quad (4)$$

In Eq. 4,  $T_{I/O}$  emerges as the dominant factor influencing the search efficiency. For instance, upon visualizing the search time costs of DiskANN on BIGANN (Fig. 11(d)), it becomes evident that  $T_{I/O}$  constitutes up to 92.5% of  $T_{total}$ , while  $T_{comp}$  and  $T_{other}$  collectively occupy less than 7.5%. This highlights the efficiency bottleneck as the cost of disk I/Os for HVSS on the disk-resident graph index. Therefore, our objective is to improve search efficiency by minimizing  $T_{I/O}$ . Note that we must not introduce additional space overhead (including memory and disk), compared to the original graph index, in order to adhere to the space capacity limitation of the data segment.

### 3 DESIGN PHILOSOPHY

We analyze two key factors that affect I/O time for existing graph index methods. Then we give an overview of Starling, illustrating its data layout and search strategy on the data segment.

#### 3.1 I/O-efficiency Analysis

When conducting searches on the disk-based graph, I/O-efficiency depends on two primary factors: the vertex utilization ratio in each disk I/O and the length of the search path. The former indicates the extent to which useful vertices are loaded in a given block, while the latter denotes the number of hops required from the entry point to the result point. Both of these factors influence the number of disk I/Os and thus determine the I/O time. A higher vertex utilization ratio signifies that more relevant information is loaded in each disk I/O, leading to more effective utilization of disk bandwidth and requiring fewer disk I/Os for each query. A longer search path implies an increased number of disk I/Os during the search process. We delve into an analysis of the vertex utilization ratio and the search path length within the *baseline framework*<sup>2</sup> and reveal two underlying issues.

**Problem 1: Poor data locality.** Fig. 2(a) shows that the baseline assigns ID-consecutive vertices (a vertex contains the vector data and neighbor IDs) to the same blocks. For example, block 0 stores vertices 0~3. Since a block is the smallest disk I/O unit, reading vertex 2 also reads vertices 0, 1, and 3. This wastes disk bandwidth as the other three vertices are irrelevant to vertex 2. A naive way to avoid the wastage is to check all the vertices in a block. However, a segment has up to  $10^7 \times$  more vertices than a block in a real-world scenario. The probability of finding a near vertex (w.r.t the target vertex) among the non-target vertices<sup>3</sup> in the block is too low to improve efficiency effectively. Therefore, the baseline only checks the target vertex and discards the rest. This means a low vertex utilization ratio in each disk I/O. According to our evaluation on BIGANN (Tab. 2), up

<sup>2</sup>Unless specifically stated, we refer to DiskANN as the baseline framework.

<sup>3</sup>We load a block according to a target vertex. In this example, the target vertex is 2.